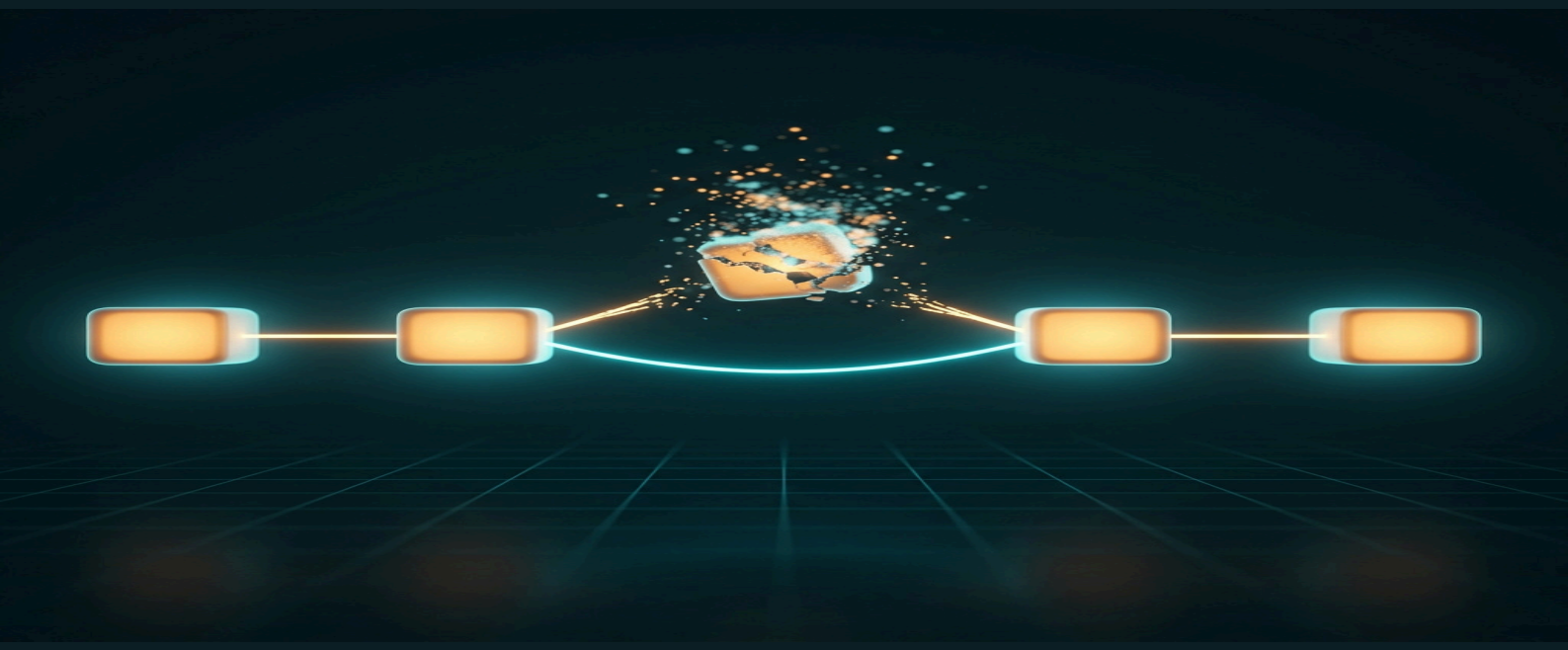


حذف عنصر من القائمة المترابطة

Linked List Deletion — شرح مبسّط بالعربية

هياكل البيانات · Data Structures · لغة C#

مُعدّ آليًا من شرائح المحاضرة المرسومة ضوئيًا
١٠ شرائح أصلية بعد فرز التكرار الناتج عن المسح

01 ما هي القائمة المترابطة؟

القائمة المترابطة سلسلة من العقد (Nodes)، كل عقدة تحمل قيمة ومؤشراً (Pointer) يشير إلى العقدة التالية. على عكس المصفوفة، لا توجد فهارس: للوصول إلى عقدة يجب السير عبر المؤشرات من الرأس (Head) حتى تصل إليها.

تشريح العقدة الواحدة:



مثال: سلسلة من ثلاث عقد



> القاعدة الذهبية: الحذف في القائمة المترابطة ليس محواً للبيانات، بل إعادة توصيل للمؤشرات — العقدة المحذوفة تُفصل عن السلسلة فتصبح معزولة.

لماذا هذا مهم؟ لأن كفاءة الحذف بعد الوصول للعقدة هي O(1): لا نحرّك أي بيانات كما في المصفوفة، نغيّر مرجعاً واحداً فقط. التحدي الحقيقي هو الوصول إلى العقدة الصحيحة والتعامل مع الحالات الحرجة.

خريطة عمليات الحذف الثلاث 02

كل عملية حذف في القائمة الأحادية تنتمي لواحدة من ثلاث حالات، والفرق بينها: من الذي يحمل المؤشر الذي سنعيد توصيله؟

المؤشر عند العقدة السابقة للهدف — نتخطى الهدف.

O(n)

المؤشر عند العقدة قبل الأخيرة — نجعله null.

O(n)

المؤشر المعني هو head نفسه — نزيحه للعقدة التالية.

O(1)

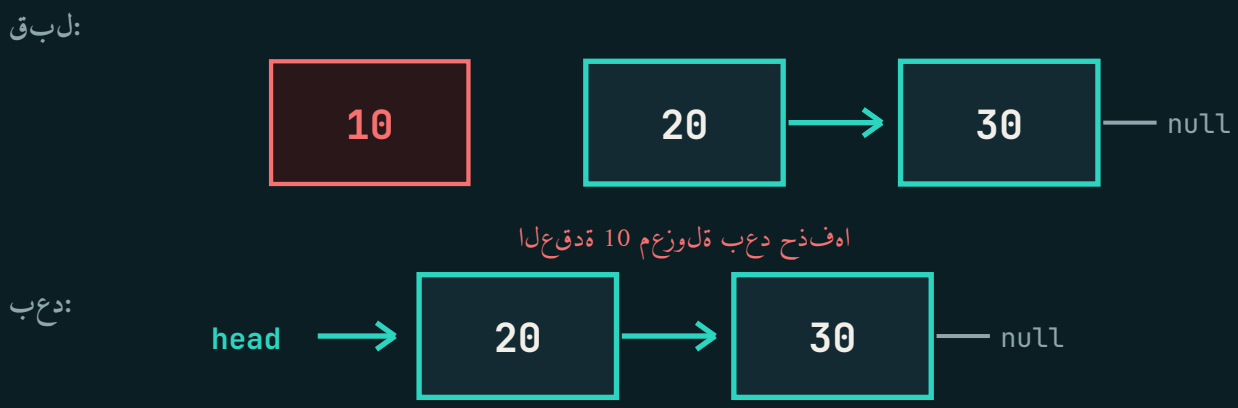
النمط الموحد: ابحث عن العقدة التي تحمل المؤشر نحو الهدف، ثم أعد توصيله ليتخطى الهدف. هذا كل شيء — ثلاث تطبيقات لفكرة واحدة.

في الصفحات التالية نفصّل كل عملية: الفكرة، ثم الخوارزمية الزائفة (Pseudocode) كما وردت في الشرائح، ثم كود C# الكامل، ثم ملاحظة الحالات الحرجة التي يحمي منها الكود.

العملية الأولى: حذف عقدة الرأس 03

الفكرة

1. تحقق أن القائمة غير فارغة (head != null).
2. أزح مؤشر head ليشير إلى head.next — بذلك تُفصل العقدة الأولى.



PSEUDOCOD

```
IF head != null:
    head = head.next
```

C#

```
Node DeleteFirst(Node head)
{
    if (head == null)
        return null;
    head = head.next;
    return head;
}
```

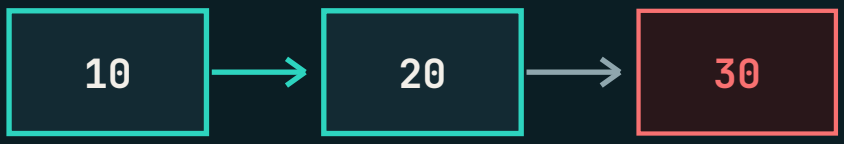
الحالة الحرجة: قائمة فارغة — الكود يعيد null مباشرة. هذه أبسط العمليات: لا حلقة ولا سير، خطوة واحدة فقط.

العملية الثانية: حذف العقدة الأخيرة 04

الفكرة

1. تحقق أن القائمة غير فارغة.
2. سِر من الرأس حتى تصل إلى العقدة قبل الأخيرة (عندما يصبح current.next.next == null).
3. اجعل next للعقدة قبل الأخيرة يساوي null — فتُقطع الأخيرة.

قبل:



30 تفقد مرجعها وتُفصل

بعد:



PSEUDOCOD

```
IF head == null: return
current = head
WHILE current.next.next != null:
    current = current.next
current.next = null
```

C#

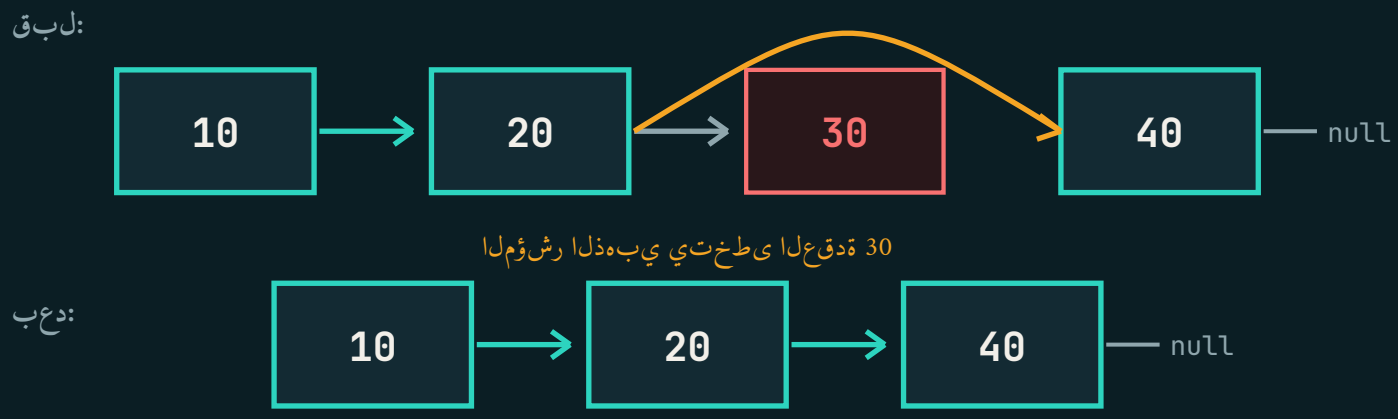
```
Node DeleteAtEnd(Node head)
{
    if (head == null || head.next == null)
        return null;
    Node current = head;
    while (current.next.next != null)
        current = current.next;
    current.next = null;
    return head;
}
```

05 انتبه للفارق: الكود يضيف الشرط **head.next == null** لمعالجة قائمة من عقدة واحدة — الشرط الذي تغفله الخوارزمية الزائفة. هنا جوهر الفرق بين الفكرة والتطبيق.

العملية الثالثة: حذف عقدة بالقيمة (المنتصف) 05

الفكرة

1. إن كان الرأس نفسه يحمل القيمة المطلوبة: احذف الرأس مباشرة.
2. وإلا: سِر حتى تجد العقدة السابقة للهدف (current.next.data == value).
3. أعد التوصيل: current.next = current.next.next — فيتخطى المؤشر الهدف.



PSEUDOCOD

```
current = head
WHILE current.next.data != target:
    current = current.next
current.next = current.next.next
```

C# –

```
Node DeleteValue(Node head, int value)
{
    if (head == null) return null;
    if (head.data == value) return head.next;
    Node current = head;
    while (current.next != null &&
           current.next.data != value)
        current = current.next;
    if (current.next != null)
        current.next = current.next.next;
    return head;
}
```

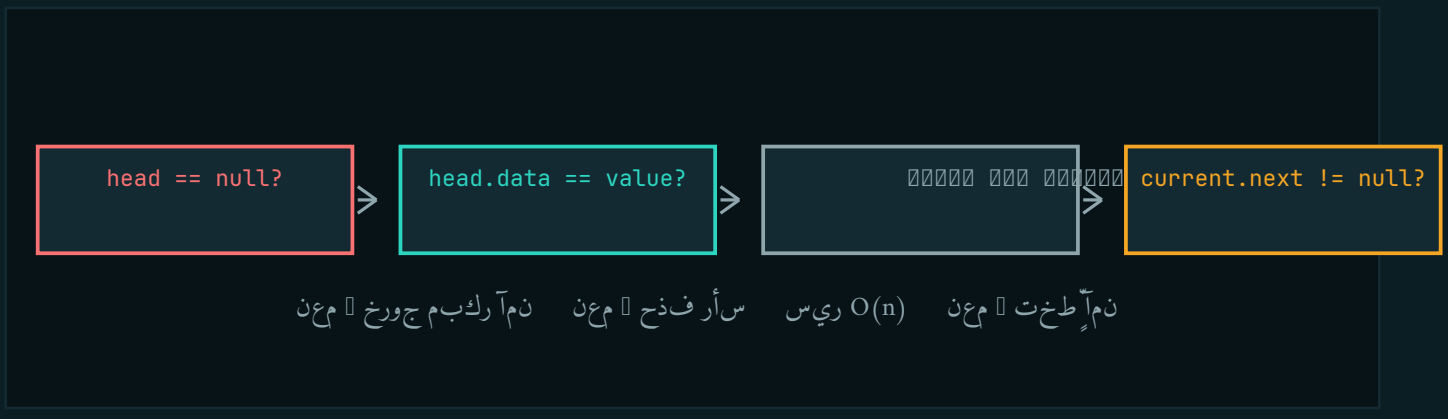
06 لماذا كود المنتصف هو الأكمل؟

دالة DeleteValue هي النسخة الأكثر نضجاً في المحاضرة: إنها تجمع حذف الرأس وحذف المنتصف في دالة واحدة، وتضيف ثلاث طبقات حماية غائبة عن الخوارزمية الزائفة:

1. قائمة فارغة: if (head == null) return null — لا محاولة وصول لمرجع معدوم.
2. الهدف هو الرأس: if (head.data == value) return head.next — معالجة خاصة لأن الرأس لا سابق له.
3. القيمة غير موجودة: الحلقة تتوقف عند null، والشرط الأخير if (current.next != null) يمنع التعديل الوهمي.

> الدرس: الخوارزمية الزائفة تعلّمك الفكرة في المسار السعيد، والكود يعلّمك حماية المسارات غير السعيدة. افهم الاثنين معاً.

قارن بنفسك: الخوارزمية الزائفة للمنتصف تفترض وجود القيمة وتفترض أن الهدف ليس الرأس — افتراضان يناهاران عند أول مدخلات حقيقية. الكود لا يفترض شيئاً.



07 جدول المقارنة الشامل

الحالة	ماذا تفعل؟	التعقيد	الخطر الشائع
حذف الرأس	إزاحة head إلى العقدة التالية	O(1)	نسيان فحص القائمة الفارغة
حذف الذيل	بلوغ العقدة قبل الأخيرة وقطع وصلتها بالأخيرة	O(n)	قائمة من عقدة واحدة
حذف بالقيمة	الوصول للسابق وتخطّي الهدف	O(n)	قيمة غير موجودة / الهدف هو الرأس

لاحظ: كل العمليات O(1) بعد الوصول — الكلفة الحقيقية هي السير. لهذا القائمة المترابطة تتفوق في الحذف المتكرر من الرأس وتتأخر عند الحذف من الذيل.

08 قائمة فحص الحالات الحرجة

قبل أن تعتبر أي دالة حذف صحيحة، اسأل نفسك هذه الأسئلة الخمسة. المحاضرة طبّقتها جميعاً في DeleteValue:

- [x] القائمة فارغة
- [x] عقدة واحدة
- [x] الهدف هو الرأس
- [x] القيمة غير موجودة
- [x] إرجاع الرأس

> خمسة أسئلة، خمس حمايات. احفظها كقائمة مرجعية قبل تسليم أي تمرين على القوائم المترابطة.

الخلاصة: الجوهر في سطرين 09

بعد خمس مرات من سؤال «لماذا»، نصل للجوهر: الحذف في القائمة المترابطة عملية إعادة توصيل — نقطع مرجعاً ونخلق مرجعاً، ولا تُمسّ البيانات. كل العمليات الثلاث تطبيقات لفكرة واحدة تختلف فقط في: من يحمل المرجع الذي سنعيد توصيله؟

> «أوجد حامل المؤشر، ثم أعد توصيله ليتخطى الهدف» — هذه الجملة تختصر المحاضرة كاملة.



تدرّب: نفّذ الدوال الثلاث على ورقة بمؤشرات مرسومة، ثم جرّب الحالات الحرجة الخمس. الفهم الحقيقي يأتي عندما تنهار القائمة بين يديك ثم تصلحها.

أُعدّ هذا الملخص وفق منهجية حلقات التفكير: فهم ← سياق ← تحليل عميق ← أساليب عرض ← تدقيق.
المصدر: ١٠ شرائح فريدة مستخلصة من ملف CamScanner الممسوح في المستودع.